

DOCUMENTACION EJERCICIO 1:

Explicación de la implementación:

Para este ejercicio del aeropuerto lo resolvimos pensando el problema como una variante clásica de "lectores y escritores": los pasajeros son lectores del cartel y los oficinistas son escritores. La idea central fue garantizar que muchos pasajeros puedan mirar el cartel al mismo tiempo, pero que cuando un oficinista esté modificando, nadie más lo pueda leer ni escribir, para evitar ver el cartel "a mitad de cambio".

En el código usamos threads POSIX para modelar tanto a los pasajeros como a los oficinistas. Creamos una cantidad fija de hilos: 100 pasajeros y 5 oficinistas. Cada hilo pasajero ejecuta una función que simula que el pasajero llega al cartel, toma el permiso de lectura, imprime el mensaje "Pasajero X está mirando el cartel", duerme un tiempo aleatorio (no mayor a 3 segundos) usando `sleep` o `usleep` con `rand()`, y luego libera el permiso de lectura. Los oficinistas hacen algo parecido pero en modo escritura: toman el permiso exclusivo, imprimen "Oficinista X está modificando el cartel", simulan el tiempo de trabajo con un `sleep` aleatorio de hasta 5 segundos, y luego sueltan el bloqueo para que otros puedan volver a leer o escribir. Cada oficinista repite este ciclo al menos 3 veces para cumplir con la consigna.

Para la sincronización definimos una estructura compartida que representa el estado del cartel y las variables de control: un mutex para proteger las variables internas y algunos contadores (por ejemplo, cantidad de lectores activos, si hay un escritor activo, cuántos escritores están esperando, etc.). Además usamos variables de condición para que los hilos puedan esperar y despertarse en el momento adecuado. Cuando un pasajero quiere leer, entra a una función tipo `inicioLectura`: toma el mutex, espera mientras haya un escritor activo o se haya decidido darle prioridad a los escritores, incrementa el contador de lectores y recién ahí suelta el mutex y "mira" el cartel. Al terminar, llama a `finLectura`, vuelve a tomar el mutex, decrementa el contador de lectores y, si quedó en cero, despierta a los escritores que estaban esperando. Los oficinistas usan funciones análogas `inicioEscritura` y `finEscritura`: el escritor espera mientras haya lectores o otro escritor activo, y cuando consigue el acceso marca que está escribiendo; al terminar limpia ese estado y despierta a lectores o escritores según la lógica que hayamos elegido.

También tuvimos que tener cuidado con la generación de los tiempos aleatorios. Inicializamos la semilla con `srand(time(NULL))` y, dentro de cada hilo, calculamos el delay como un valor aleatorio acotado. De esa forma cada hilo se comporta de forma independiente y el orden de los mensajes en pantalla va cambiando de ejecución en ejecución, lo cual ayuda a comprobar que la sincronización realmente está funcionando y que nunca aparece un pasajero "mirando el cartel" mientras un oficinista está "modificando el cartel".

Finalmente, en el main montamos toda la simulación: inicializamos el mutex y las variables de condición, creamos los hilos de pasajeros y oficinistas con `pthread_create`, y al final hacemos `pthread_join` para esperar a que terminen sus tareas antes de liberar los recursos y terminar el programa. En resumen, la solución se apoyó en modelar correctamente la relación lectores/escritor y usar las primitivas de sincronización de POSIX para asegurar que el cartel siempre se vea coherente, sin que un pasajero pueda leer mientras algún oficinista está en medio de una actualización.